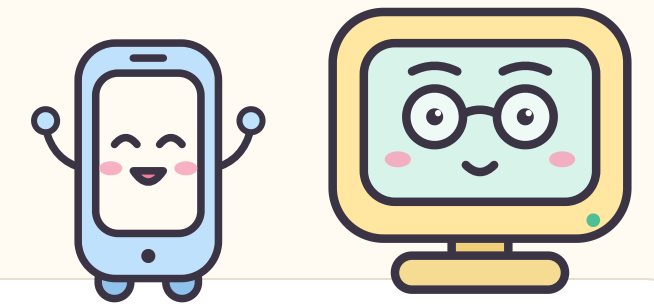




第 14 集 · 约 23 分钟

服务端⑦：分块传输

传到一半断网了，接着传

14

从零写出 LanShare

这一集你会学到

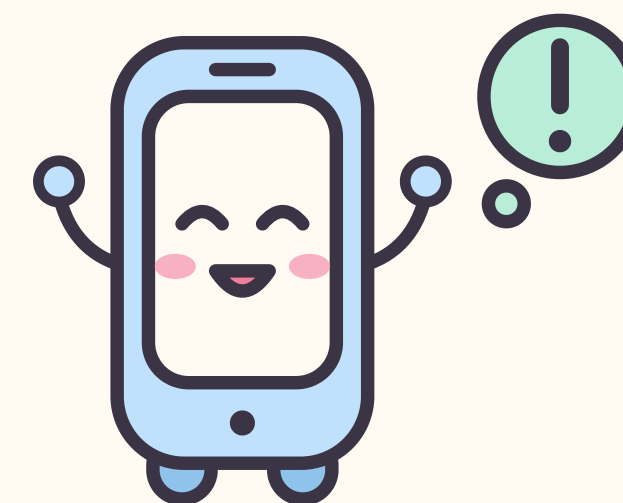
1. 上传的一生：begin 预占空间 → 分块按偏移写入 → status 续传 → finish 改名落盘
2. spawn_blocking 加 move 闭包：阻塞的文件操作交给别的线程，所有权一起交过去
3. 上传名额在同一次加锁里检查并占位；预占后磁盘还要留余量（审查 I-2、复验 N-5）
4. Weak 弱引用：后台清理任务不拖住服务的生命
5. 下载的第一个字节是状态；checked_mul 防止块号乘法溢出（审查 M-5）



先想一个问题

一个 2 GB 的视频，传到 90% 的时候 WiFi 断了一下。接下来怎么办？

v1 只能从头再来。v2 问一句“你收到了哪些块”，把剩下的补上。





一次上传的一生



发生了什么

磁盘不够，这一步就失败

每块写到“块号 × 1 MiB”的位置

断网之后，问收到了哪些



审查 I-2：没传完的上传

○ 没传完的上传永不过期

一共 64 个名额，正常使用中半途而废的上传就会慢慢把它们耗光

○ 检查名额有竞态

审查实测：同一会话并发 100 个 upload_begin，全部成功

○ 预占磁盘不留余量

把磁盘占满，电脑上别的程序也跟着出问题

● 修法：10 分钟没有新块就作废；名额在同一次加锁里查并占位；预占后至少还剩 512 MiB；单个文件最大 1 TiB



transfer.rs: 开头

transfer.rs

crates/lanshare/src/server/

第 3-22 行 · 共 22 行

```

3  ///! 上传: `upload_begin` 建一个隐藏临时文件 → 每块按偏移写入 (可并发、可重发、可续传) →
4  ///! `upload_finish` 收齐后改名落盘。上传只属于发起它的会话, 别的设备碰不到。
5  ///! 下载: 每块单独请求; 响应明文第一个字节是状态 (0 = 后面是数据, 1 = 后面是 JSON 错误)。
6  ///!
7  ///! 资源上限 (独立安全审查 I-2): 全局和每个会话的并发上传数在**同一次加锁**里检查并占位;
8  ///! 一段时间没有新块的上传自动作废 (删临时文件、释放预占的磁盘空间); 预占空间前确认磁盘还留有余量。
9
10 use std::collections::HashMap;
11 use std::fs::{self, OpenOptions};
12 use std::io::{self, Read, Seek, SeekFrom, Write};
13 use std::net::IpAddr;
14 use std::path::PathBuf;
15 use std::sync::{Arc, Weak};
16 use std::time::{Duration, Instant};
17
18 use lanshare_proto::{CHUNK, sanitize_filename};
19 use serde_json::{Value, json};
20
21 use super::rpc::{reply, reply_error};
22 use super::{AppState, random_hex};

```



transfer.rs: 上传记录和几个小工具

● ● ● transfer.rs crates/lanshare/src/server/ 第 50-69 行 • 共 69 行

```
50         io::ErrorKind::StorageFull => "disk_full",
51         io::ErrorKind::NotFound => "not_found",
52         _ => "io_error",
53     }
54 }
55
56 pub(crate) fn human_size(n: u64) -> String {
57     const UNITS: [&str; 4] = ["KB", "MB", "GB", "TB"];
58     if n < 1024 {
59         return format!("{n} B");
60     }
61     let mut v = n as f64;
62     for unit in UNITS {
63         v /= 1024.0;
64         if v < 1024.0 {
65             return format!("{v:.1} {unit}");
66         }
67     }
68     format!("{v:.1} PB")
69 }
```



human_size: v1 和 v2

Python v1 • server.py

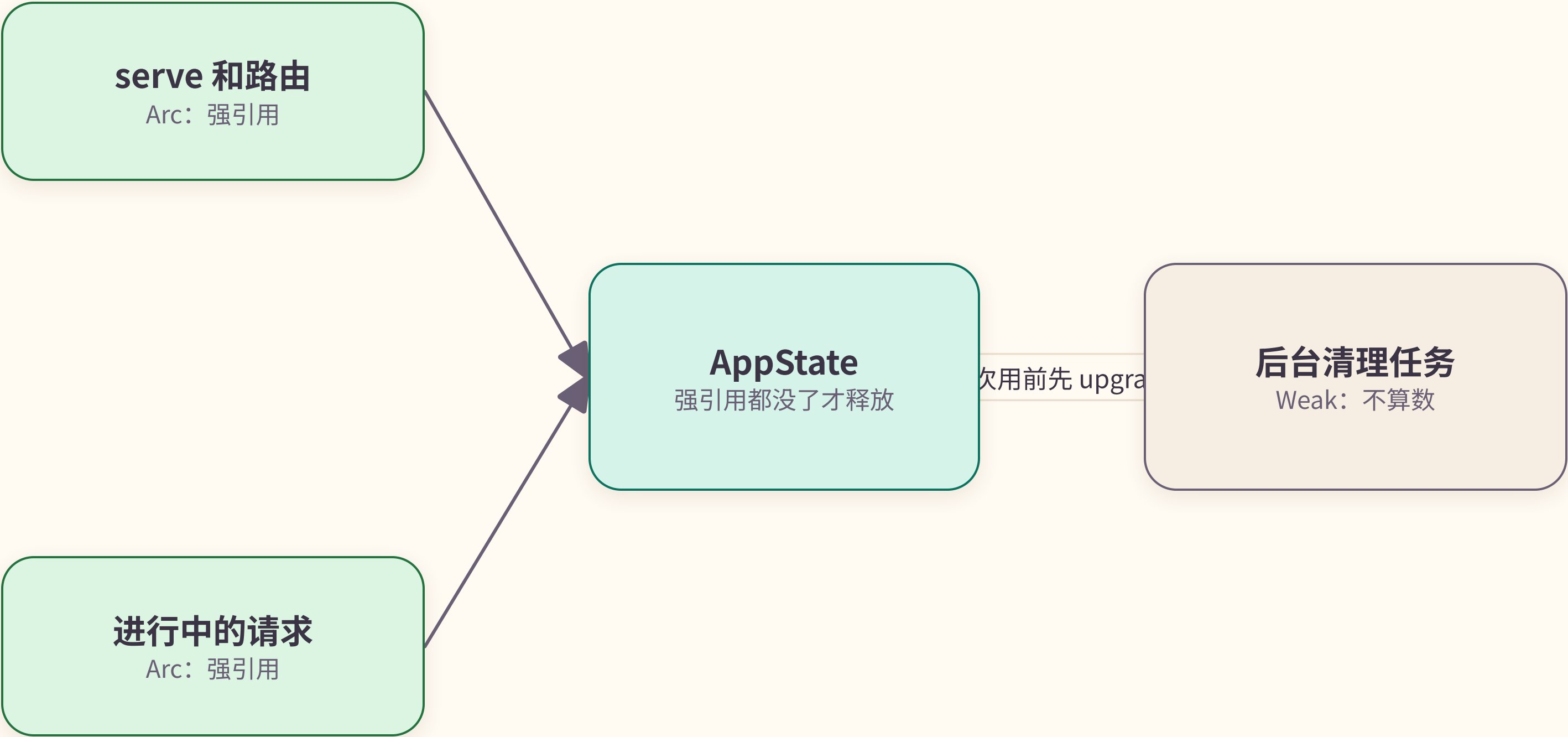
```
1 def human_size(n):
2     if n < 1024:
3         return "%d B" % n
4     for unit in ("KB", "MB", "GB"):
5         n /= 1024.0
6         if n < 1024:
7             return "%.1f %s" % (n, unit)
8     return "%.1f TB" % (n / 1024.0)
```

Rust v2 • transfer.rs

```
1 pub(crate) fn human_size(n: u64) -> String {
2     const UNITS: [&str; 4] = ["KB", "MB", "GB", "TB"];
3     if n < 1024 {
4         return format!("{n} B");
5     }
6     let mut v = n as f64;
7     for unit in UNITS {
8         v /= 1024.0;
9         if v < 1024.0 {
10             return format!("{v:.1} {unit}");
11         }
12     }
13     format!("{v:.1} PB")
14 }
```



强引用和弱引用





transfer.rs: 过期清理

transfer.rs crates/lanshare/src/server/

第 83-102 行 • 共 102 行

```
83     }
84     stale.len()
85 }
86
87 /// 后台定时清理。只持有弱引用：服务停了、状态释放了，这个任务也就结束了。
88 pub(crate) async fn reap_forever(state: Weak<AppState>) {
89     loop {
90         let Some(st) = state.upgrade() else { return };
91         let every = (st.upload_idle / 2).clamp(Duration::from_millis(100), Duration::from_secs(30));
92         drop(st);
93         tokio::time::sleep(every).await;
94         let Some(st) = state.upgrade() else { return };
95         let removed = tokio::task::spawn_blocking(move || reap(&st)).await.unwrap_or(0);
96         if removed > 0
97             && let Some(st) = state.upgrade()
98         {
99             st.log(&format!("清理了 {removed} 个长时间没有动静的上传"));
100         }
101     }
102 }
```



transfer.rs: upload_begin (上)

transfer.rs crates/lanshare/src/server/

第 131–150 行 • 共 150 行

```
131         last_active: Instant::now(),
132     };
133     uploads.0.insert(upload_id.clone(), upload);
134 }
135
136 let st2 = st.clone();
137 let temp2 = temp.clone();
138 let reserve = st.disk_reserve;
139 let created = tokio::task::spawn_blocking(move || -> io::Result<()> {
140     st2.storage.ensure_root()?;
141     // 查剩余空间和预占空间一个接一个做：并发的 begin 不会看到同一个剩余空间（复验 N-5）
142     let _alloc = st2.alloc_lock.lock().unwrap_or_else(|e| e.into_inner());
143     if st2.storage.available_space().is_some_and(|free| free < size.saturating_add(reserve)) {
144         return Err(io::Error::from(io::ErrorKind::StorageFull));
145     }
146     let file = fs::File::create(&temp2)?;
147     file.set_len(size)?; // 预先占好空间：磁盘不够时这里就会失败，而不是传到一半
148     Ok(())
149 })
150 .await;
```



transfer.rs: upload_begin (下)

transfer.rs crates/lanshare/src/server/

第 143–162 行 • 共 162 行

```
143         if st2.storage.available_space().is_some_and(|free| free < size.saturating_add(reserve)) {
144             return Err(io::Error::from(io::ErrorKind::StorageFull));
145         }
146         let file = fs::File::create(&temp2)?;
147         file.set_len(size)?; // 预先占好空间: 磁盘不够时这里就会失败, 而不是传到一半
148         Ok(())
149     })
150     .await;
151     let failed = match created {
152         Ok(Ok(())) => None,
153         Ok(Err(e)) => Some(io_error_code(&e)),
154         Err(_) => Some("io_error"),
155     };
156     if let Some(code) = failed {
157         st.uploads.lock().expect("锁不会中毒").0.remove(&upload_id);
158         let _ = fs::remove_file(&temp);
159         return reply_error(code);
160     }
161     reply(json!({ "ok": true, "upload_id": upload_id, "chunk": CHUNK }))
162 }
```



transfer.rs: 查进度、完成、放弃

transfer.rs crates/lanshare/src/server/

第 196-215 行 · 共 215 行

```

196         Ok(Err(e)) => {
197             let _ = fs::remove_file(&upload.temp);
198             reply_error(io_error_code(&e))
199         }
200         Err(_) => reply_error("io_error"),
201     }
202 }
203
204 pub(crate) fn abort(st: &AppState, request: &Value, cid: [u8; 16]) -> Vec<u8> {
205     let mut uploads = st.uploads.lock().expect("锁不会中毒");
206     let Some(id) = request["upload_id"].as_str() else { return reply_error("bad_request") };
207     match uploads.0.get(id) {
208         Some(u) if u.owner == cid => {
209             let upload = uploads.0.remove(id).expect("刚确认过存在");
210             let _ = fs::remove_file(upload.temp);
211             reply(json!({ "ok": true }))
212         }
213         _ => reply_error("not_found"),
214     }
215 }

```



按偏移写入：Python 和 Rust

Python

```
1 with open(temp, "r+b") as f:      # 打开已有文件，可写
2     f.seek(index * CHUNK)
3     f.write(data)
4 # with 结束，文件自动关闭
```

Rust

```
1 let mut file = OpenOptions::new().write(true).open(&temp)?;
2 file.seek(SeekFrom::Start(index as u64 * CHUNK as u64))?;
3 file.write_all(&data)
4 // file 离开作用域，自动关闭
```



transfer.rs: 写一块

transfer.rs crates/lanshare/src/server/

第 235-254 行 • 共 254 行

```
235     }
236     let written = tokio::task::spawn_blocking(move || -> io::Result<()> {
237         let mut file = OpenOptions::new().write(true).open(&temp)?;
238         file.seek(SeekFrom::Start(index as u64 * CHUNK as u64))?;
239         file.write_all(&data)
240     })
241     .await;
242     match written {
243         Ok(Ok(())) => {
244             let mut uploads = st.uploads.lock().expect("锁不会中毒");
245             if let Some(upload) = uploads.0.get_mut(upload_id) {
246                 upload.received[index] = true;
247                 upload.last_active = Instant::now();
248             }
249             reply(json!({ "ok": true }))
250         }
251         Ok(Err(e)) => reply_error(io_error_code(&e)),
252         Err(_) => reply_error("io_error"),
253     }
254 }
```



transfer.rs: 读一块

transfer.rs crates/lanshare/src/server/

第 289-308 行 · 共 308 行

```

289         return Err("bad_index");
290     }
291     let len = if size == 0 { 0 } else { expected_len(size, index) };
292     let mut out = vec![0u8; 1 + len as usize];
293     file.seek(SeekFrom::Start(start)).map_err(|e| io_error_code(&e))?;
294     file.read_exact(&mut out[1..]).map_err(|e| io_error_code(&e))?;
295     Ok(out)
296 })
297 .await;
298 match result {
299     Ok(Ok(out)) => {
300         if index == 0 {
301             st.log(&format!("发送 {name} 到 {peer}"));
302         }
303         out
304     }
305     Ok(Err(code)) => down_error(code),
306     Err(_) => down_error("io_error"),
307 }
308 }

```



mod.rs: 上传表和后台清理

```
mod.rs crates/lanshare/src/server/ 第 390-409 行 • 共 443 行

390 tokio::spawn(transfer::reap_forever(Arc::downgrade(&state)));
391 let app = router(state.clone());
392 let admission = conn::Admission::new(state.max_connections, state.max_local_connections, state.max_connections_per_ip);
393 let graceful = hyper_util::server::graceful::GracefulShutdown::new();
394 let mut shutdown = std::pin::pin!(shutdown);
395 loop {
396     let accepted = tokio::select! {
397         accepted = listener.accept() => accepted,
398         _ = &mut shutdown => break,
399     };
400     let (stream, peer) = match accepted {
401         Ok(pair) => pair,
402         // 对方刚连上就断了之类的连接级错误：直接接下一个（和 axum 自带的处理一样，第三轮复验 R3-3）
403         Err(e) if is_connection_error(&e) => continue,
404         // 句柄耗尽之类的错误：稍等再接；等的时候也响应“退出”
405         Err(_) => tokio::select! {
406             _ = tokio::time::sleep(Duration::from_millis(50)) => continue,
407             _ = &mut shutdown => break,
408         },
409     };
};
```



channel.rs: 接上上传和下载

channel.rs

crates/lanshare/src/server/

第 130-149 行 · 共 149 行

```

130         Ok(Ok(body)) => body,
131         Ok(Err(_)) => return StatusCode::PAYLOAD_TOO_LARGE.into_response(),
132         Err(_) => return StatusCode::REQUEST_TIMEOUT.into_response(),
133     };
134     let opened = match open_body(&st, cid, ctr, &keys, &path, &body) {
135         Ok(opened) => opened,
136         Err(status) => return status.into_response(),
137     };
138     let local = peer.ip().is_loopback();
139     let segments: Vec<&str> = path.trim_start_matches('/').split('/').collect();
140     let payload: Vec<u8> = match segments.as_slice() {
141         ["api", "rpc"] => rpc::handle(&st, &opened.plain, local, opened.cid, peer.ip()).await,
142         ["api", "up", upload, index] => {
143             transfer::put_chunk(&st, upload, index, opened.plain.clone(), opened.cid).await
144         }
145         ["api", "down", file, index] => transfer::get_chunk(&st, file, index, &opened.plain, peer.ip()).await,
146         _ => rpc::reply_error("not_found"),
147     };
148     sealed_response(&opened, &path, &payload)
149 }

```



rpc.rs: 接上上传的四个操作

● ● ●

rpc.rs

crates/lanshare/src/server/

第 45-64 行 · 共 119 行

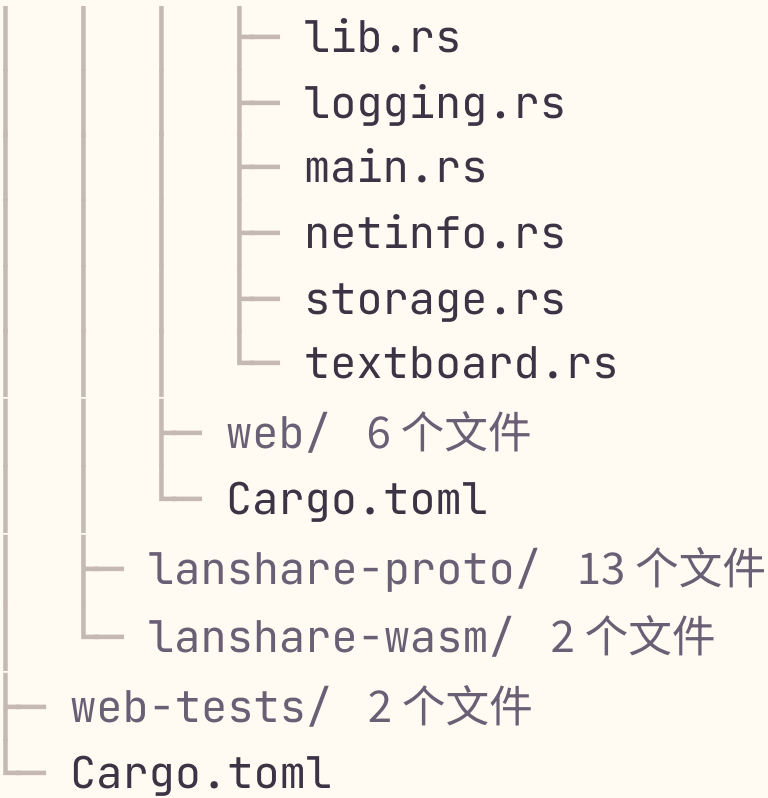
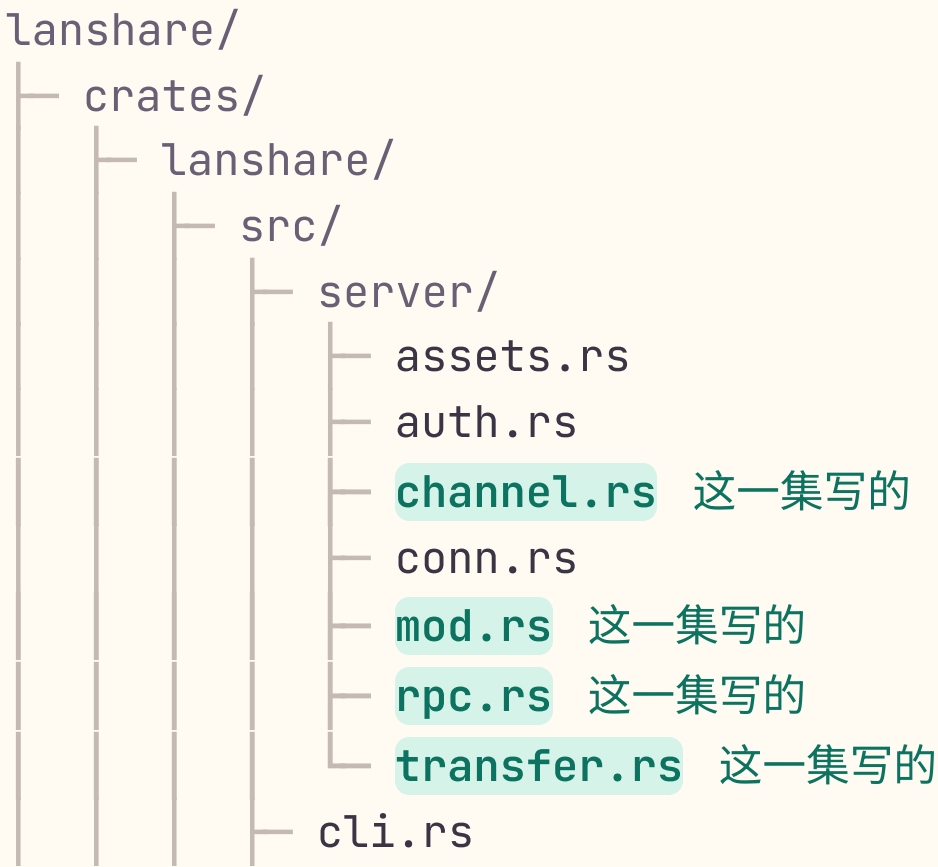
```

45     "upload_begin" => transfer::begin(st, &request, cid).await,
46     "upload_status" => transfer::status(st, &request, cid),
47     "upload_finish" => transfer::finish(st, &request, cid, peer).await,
48     "upload_abort" => transfer::abort(st, &request, cid),
49     _ => reply_error("unknown_op"),
50 }
51 }
52
53 fn info(st: &AppState, local: bool) -> Vec<u8> {
54     let mut info = json!({
55         "ok": true,
56         "version": env!("CARGO_PKG_VERSION"),
57         "server_id": st.server_id(),
58         "urls": st.base_urls(),
59         "pin": st.pin(),
60         "pin_paused": st.pin_paused(),
61         "qr_svg": qr_svg(&st.entry_url()),
62         // 已配对设备（包括用口令配对的）经加密信道拿到 K，下次打开页面可直接握手
63         "key": st.key_fragment(),
64         "local": local,

```



服务端写齐了





编译、跑测试

```
Windows PowerShell

PS D:\code\lanshare> cargo build
  Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Compiling lanshare-wasm v2.0.0 (D:\code\lanshare\crates\lanshare-wasm)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 3.07s

PS D:\code\lanshare> cargo test -p lanshare --lib
  Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
  Finished `test` profile [unoptimized + debuginfo] target(s) in 1.90s
  Running unittests src\lib.rs (target\debug\deps\lanshare-c9b4a6a3f956eda2.exe)

running 23 tests
..... (省略 23 行, 和上一集一样, 全部 ok)

test result: ok. 23 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.02s
```





选择题

下面哪个操作会刷新上传的“最后活动时间”，让它不被清理？

A 成功写入一块

B 查一次进度 `upload_status`

C 收到一个块号越界的请求



选择题

下面哪个操作会刷新上传的“最后活动时间”，让它不被清理？

A 成功写入一块

put_chunk 写块前后都刷新活动时间。只有真的在传数据，上传才算活着。

B 查一次进度 upload_status

复验 N-5 之后，查进度只读不刷新，否则只查不传就能让上传永远占着空间。

C 收到一个块号越界的请求

块号越界在刷新之前就返回 bad_index 了。



选择题

reap_forever 为什么拿 Weak，而不是 Arc？

- A Weak 比 Arc 快
- B 后台任务不该拖住状态：服务停了，状态要能释放，任务也随之结束
- C Arc 不能跨线程



选择题

reap_forever 为什么拿 Weak，而不是 Arc？

A Weak 比 Arc 快

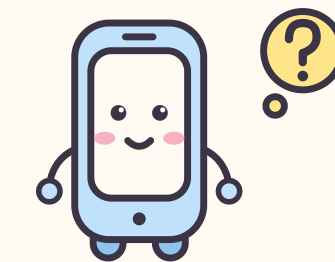
速度不是原因，每次用之前还要多做一次 upgrade。

C Arc 不能跨线程

Arc 正是为跨线程共享设计的。

B 后台任务不该拖住状态：服务停了，状态要能释放，任务也随之结束

强引用会让 AppState 永远释放不掉。弱引用不算数，所有强引用都没了，upgrade 返回 None，任务自己退出。



选择题

put_chunk 里，块号乘以 1 MiB 直接用了乘号；get_chunk 里却用 checked_mul。为什么？

- A put_chunk 先查过块号小于块数，乘出来不会超过文件大小；get_chunk 在乘之前没有这道检查
- B 写文件不会溢出，读文件才会
- C 两个都应该用乘号



选择题

put_chunk 里，块号乘以 1 MiB 直接用了乘号；get_chunk 里却用 checked_mul。为什么？

A put_chunk 先查过块号小于块数，乘出来不会超过文件大小；get_chunk 在乘之前没有这道检查

上传的块号在锁里和 received 的长度比过，乘积最多 1 TiB 左右。下载的块号直接来自网络，乘之前没有约束，超大的数会溢出，所以用 checked_mul（审查 M-5）。

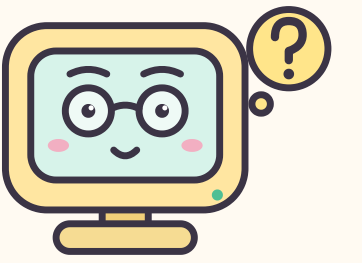
B 写文件不会溢出，读文件才会

溢出是乘法本身的事，和读写无关。

C 两个都应该用乘号

debug 构建里溢出会直接崩溃，release 里会悄悄绕回去，都不该发生。

想一想



**begin 里，查剩余空间和预占空间为什么要放进 alloc_lock?
名额已经在 uploads 的锁里占过了，还不够吗？**



想一想

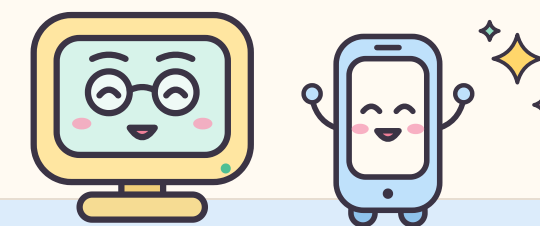
**begin 里，查剩余空间和预占空间为什么要放进 alloc_lock?
名额已经在 uploads 的锁里占过了，还不够吗？**

名额管的是“同时有几个上传”，不管每个上传有多大。两个并发的 begin 都拿到了名额，然后各自去查剩余空间：如果查和占不排队，它们会看到同一个剩余空间，都以为够，结果一起把磁盘占过头。所以“查剩余空间、预占”这两步也要串起来做，一次只有一个。这是复验 N-5。为什么不直接放进 uploads 的锁里？因为建文件、预占空间是磁盘操作，可能很慢，不能拿着全局的上传表锁去做，别的上传的每一块都要用那把锁。



这一集写了什么

- ✓ 分块上传：begin 预占空间，块按偏移写入、可乱序、可重发，status 续传，finish 不覆盖地改名
- ✓ spawn_blocking 加 move：阻塞的文件操作交给别的线程，数据的所有权一起交过去
- ✓ 名额在同一次加锁里查并占位；10 分钟没动静就作废；预占后至少留 512 MiB（审查 I-2、复验 N-5）
- ✓ Weak 让后台清理任务不拖住状态
- ✓ 下载第一个字节是状态；块号乘法用 checked_mul（审查 M-5）



下一集

全课完结，感谢收看